Trading Signal Radar — Project Proposal

Oscar Moulet · October 7, 2026

Multi-criteria trading alerts and market monitor built on n8n.

Stack: n8n (Backend), Supabase Postgres (Database), Alpha Vantage (Market Data API), AI API (Model to be confirmed), Telegram.
Target: Level 3.

1. The Need

Data-focused financial analysts and trading enthusiasts need a way to track converging technical indicators across multiple assets automatically; today, they spend hours manually refreshing charting software and logging price actions into complex Excel models to avoid missing entry points.

2. The Trigger

The primary production trigger is a Schedule Trigger running periodically (e.g., every 15 minutes) to evaluate market data. This is complemented by a Telegram Trigger that fires when you send a specific command like `/status` or `/check`. Because you cannot debug what you cannot repeat, a manual trigger will be wired next to the schedule so you can fire the workflow on demand while building.

3. The Architecture Diagram

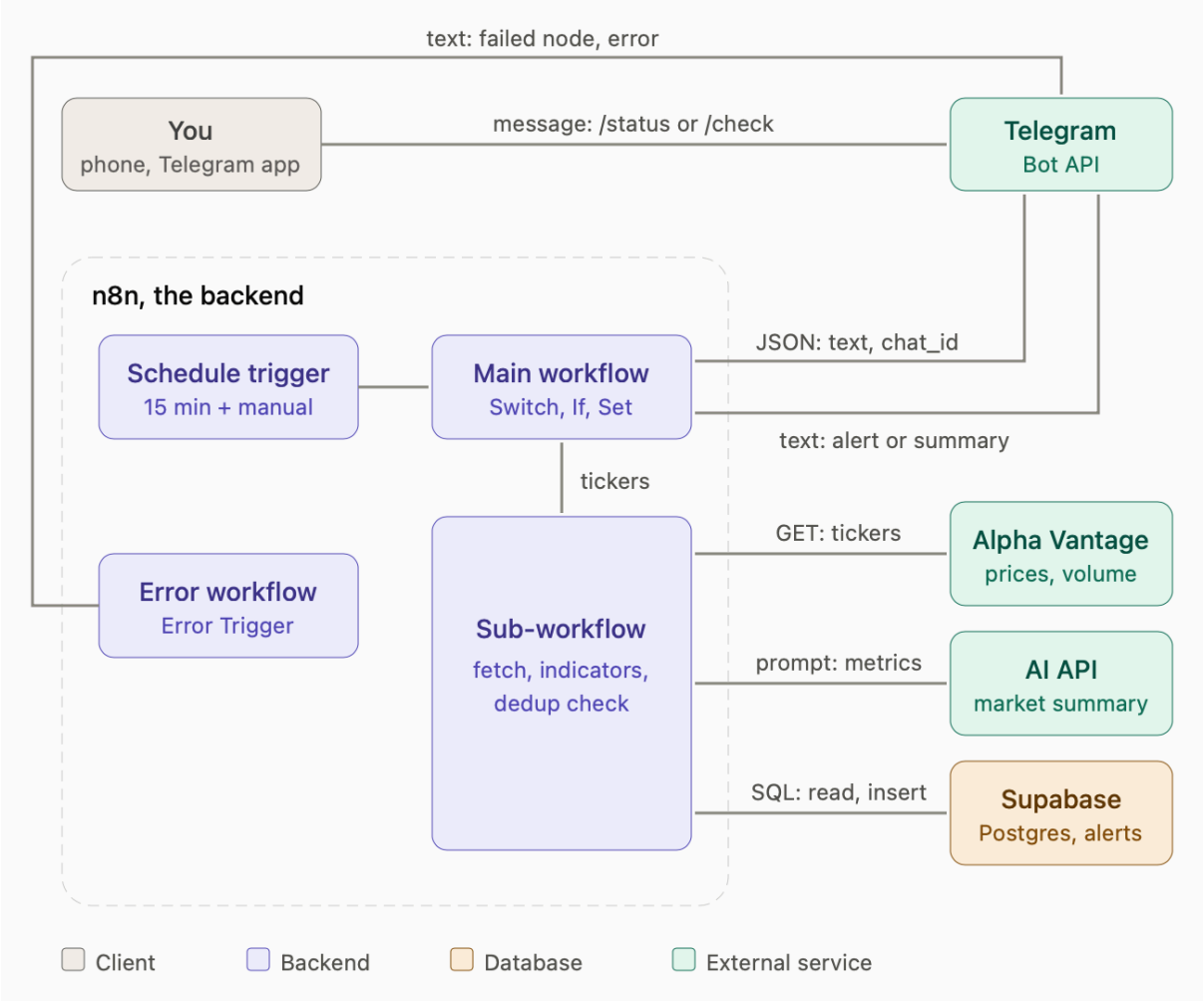
Following the rule that boxes run and arrows move data, here is the multi-stage architecture:

Blocks:

- **Backend (n8n):** The central orchestration layer handling loops, data transformations, and indicator logic.
- **Database (Postgres/Supabase):** Stores historical alert states and logs sent signals to handle the deduplication requirement.
- **External Service 1 (Market Data API):** A financial API (like Alpha Vantage) serving the raw asset prices.
- **External Service 2 (AI API):** An LLM provider like Groq or OpenAI for parsing complex indicator cross-overs into a readable summary.
- **External Service 3 (Telegram):** The chat channel where you receive the deduplicated alerts.

Arrows (pointing the direction of the request):

- **Telegram → Backend:** Webhook sends JSON containing the command text and your `chat_id`.
- **Backend → Database:** Requests to read recent alert history to check for duplicates.
- **Database → Backend:** Returns JSON rows of previously triggered alerts.
- **Backend → Market Data API:** HTTP GET requesting current data for a loop of asset tickers.
- **Market Data API → Backend:** Returns JSON arrays of price and volume data.
- **Backend → AI API:** Sends a prompt containing the technical parameters to generate a market summary.
- **AI API → Backend:** Returns the text summary.
- **Backend → Database:** SQL Insert to save the new alert state so it doesn't fire again tomorrow.
- **Backend → Telegram:** Sends the final consolidated JSON message payload to your device.



4. The Level and Cut Line

This is targeting Level 3, as it involves multi-criteria alerts, state tracking, and deduplication. If time runs out, the first cut will be the AI summarization node and the database deduplication loop. I would fall back to a Level 2 workflow that simply fetches the market

data, calculates one indicator, and pushes the alert without keeping history. The error workflow and sub-workflows would remain to satisfy the technical requirements.

5. Minimum Technical Requirements Mapping

- **If or Set node:** An If node checks if the calculated indicator crosses your target threshold; a Set node formats the final text payload before sending it to Telegram.
- **Code node:** Executes data science logic to parse the raw time-series JSON from the API and calculate technical indicators like moving averages or RSI.
- **Router or If/Switch statement:** A Switch node acts as a command router for incoming Telegram messages, directing the flow differently for `/status`, `/alert`, or invalid text.
- **API or MCP connection:** Standard HTTP Request nodes to pull the market data and interact with the AI API.
- **Message trigger:** A Telegram Trigger node listening for your commands.
- **Database for data persistence:** A relational database (like Supabase Postgres) to log which alerts have already been sent, satisfying the "keep history" and "dedup" requirements.
- **AI node:** An LLM Chain used to instantly synthesize the raw indicator metrics into a concise, human-readable market brief.
- **Sub-workflow:** The core "fetch data and evaluate indicator" logic is placed in a sub-workflow, allowing it to be called by both the nightly Schedule Trigger and the on-demand Telegram Trigger.
- **Error handling and retries:** An Error Trigger acts as a safety net to alert you on Telegram if the primary workflow fails (e.g., due to an API timeout), and the HTTP Request nodes will have built-in retry policies enabled.